

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

A DISTANCIA

PROGRAMACIÓN III

Trabajo Final - Food Store

Sistema de Gestión de Pedidos de Comida

Frontend Web + JPA + Hibernate + H2 — Web App y Menú de Consola

Se propone desarrollar un sistema de gestión de pedidos de comida llamado **Food Store**, implementado en dos partes independientes: un frontend web con TypeScript, Vite , y un backend de consola con Java, JPA/Hibernate y base de datos H2 en archivo. El sistema permite gestionar categorías, productos, usuarios y pedidos con sus líneas de detalle, incorporando relaciones JPA, soft delete, consultas JPQL personalizadas y un menú de consola navegable.

El desarrollo se organiza en dos partes complementarias. La Parte 1 desarrolla la interfaz web del sistema, consumiendo datos desde archivos .json locales mediante fetch(), de modo que todos los flujos de usuario puedan construirse y verificarse de forma independiente al backend, para en una iteración posterior reemplazar los fetch a .json por llamadas a la API REST correspondiente. La Parte 2 aborda la capa de persistencia y lógica de negocio mediante JPA + Hibernate, con un menú de consola que permite realizar operaciones CRUD sobre todas las entidades del modelo.

Tabla de Contenidos

Parte 1 – Frontend Web

- F1. Objetivo del Frontend
- F2. Fuente de Datos: Archivos JSON
- F3. Estructura del Proyecto Frontend
- F4. Autenticación y Roles
- F5. Módulos y Funcionalidades
- F6. Flujos de Usuario
- F7. Consideraciones Importantes
- F8. Historias de Usuario – Frontend

Parte 2 – Backend JPA / Consola

- 1. Objetivo General
- 2. Modelo de Dominio
 - 2.1 Entidades
 - 2.2 Relaciones entre entidades
 - 2.3 Enumerados
- 3. Estructura del Proyecto
- 4. Componentes del Sistema
 - 4.1 JPAUtil
 - 4.2 BaseRepository<T>
 - 4.3 CategoriaRepository
 - 4.4 ProductoRepository
 - 4.5 UsuarioRepository
 - 4.6 PedidoRepository
- 5. Menú de Consola
- 6. Reglas Técnicas
- 7. Historias de Usuario – Backend
- 8. Criterios de Evaluación
- 9. Condiciones de Entrega

PARTE 1

Frontend Web — Food Store Web App

F1. Objetivo del Frontend

Desarrollar la interfaz web del sistema Food Store utilizando TypeScript, Vite, HTML5, CSS3 y Tailwind CSS. En esta primera iteración, el frontend consumirá datos desde archivos .json locales mediante fetch(). Esto permite construir y verificar todos los flujos de usuario de forma independiente al backend, para en una iteración posterior reemplazar los fetch a .json por llamadas a la API REST del backend Java desarrollado en la Parte 2.

Tecnologías Frontend: TypeScript, Vite, HTML5, CSS3, Tailwind CSS.

Autenticación: gestión básica con localStorage (solo fines educativos).

F2. Fuente de Datos: Archivos JSON

En lugar de consumir la API REST, el frontend obtiene todos los datos a través de fetch() sobre archivos .json ubicados en la carpeta public/data/. Cada archivo representa un recurso del sistema.

Archivo	Contenido
public/data/categorias.json	Array de objetos Categoria: id, nombre, descripcion, imagen, eliminado.
public/data/productos.json	Array de objetos Producto: id, nombre, precio, descripcion, stock, imagen, disponible, eliminado, categoriaId.
public/data/usuarios.json	Array de objetos Usuario: id, nombre, apellido, mail, celular, password, rol. (El campo password se usa solo para verificar el login; nunca se muestra en la interfaz ni se persiste en el registro de esta iteración.)
public/data/pedidos.json	Array de objetos Pedido: id, fecha, estado, total, formaPago, idUsuario, detalles (array con idProducto, cantidad, subtotal).

Importante: cada fetch() debe apuntar a estos archivos locales. Cuando en el futuro se integre el backend, bastará con reemplazar la URL del fetch por el endpoint correspondiente de la API REST (ej: fetch('/api/products') en lugar de fetch('/data/productos.json')). La estructura del JSON debe aproximarse a la que retornaría la API. Nota: el JSON usa identificadores planos para las relaciones (categoriaId en Producto, idUsuario e idProducto en Pedido y sus detalles) mientras que el backend modela esas relaciones como objetos (Categoria, Usuario, Producto). El campo createdAt de Base puede omitirse en estos JSON de prueba; al conectar la API real, el mapeo entre ambos formatos se resuelve en la capa de serialización.

Ejemplo de fetch a JSON local:

```
// Obtener catálogo de productos desde el JSON local
const response = await fetch('/data/productos.json');
```

```
const productos = await response.json();  
// En la iteración siguiente se reemplaza por:  
// const response = await fetch('/api/products');
```

F3. Estructura del Proyecto Frontend

```
final-prog3/  
├─ index.html           # Redirección a login  
├─ package.json         # Dependencias y scripts  
├─ tsconfig.json        # Configuración TypeScript  
├─ vite.config.ts       # Configuración Vite  
└─ src/  
    ├─ main.ts          # Punto de entrada  
    ├─ style.css         # Estilos globales  
    ├─ types/           # Definiciones de tipos TypeScript  
    ├─ utils/           # Utilidades y helpers (fetch, auth, cart)  
    └─ pages/  
        ├─ auth/        # Login y registro  
        │   ├─ login/  
        │   └─ register/  
        ├─ store/       # Páginas del cliente  
        │   ├─ home/    # Catálogo de productos  
        │   ├─ productDetail/  
        │   └─ cart/    # Carrito de compras  
        ├─ client/      # Área del cliente  
        │   └─ orders/  # Mis pedidos  
        └─ admin/       # Panel de administración  
            ├─ adminHome/ # Dashboard  
            ├─ categories/ # CRUD categorías  
            ├─ products/  # CRUD productos  
            └─ orders/    # Gestión de pedidos
```

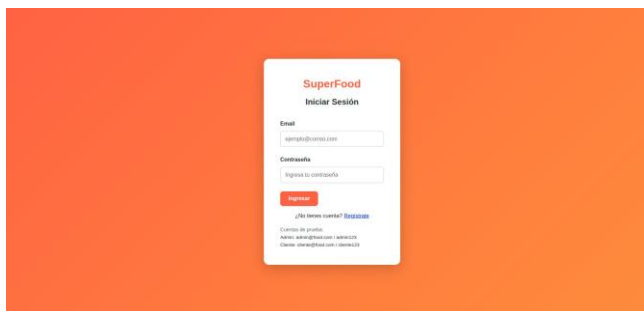
Adicionalmente, los archivos de datos deben ubicarse en:

```
public/  
└─ data/  
    ├─ categorias.json  
    ├─ productos.json  
    ├─ usuarios.json  
    └─ pedidos.json
```

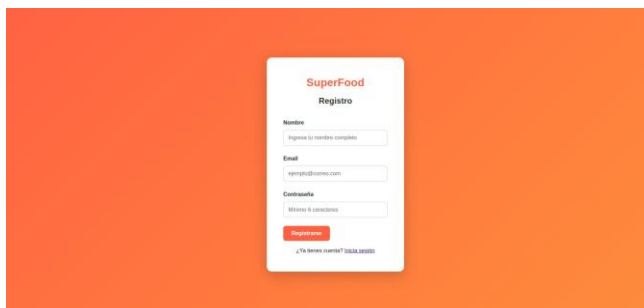
Diseño y UX

Pantallas de Referencia

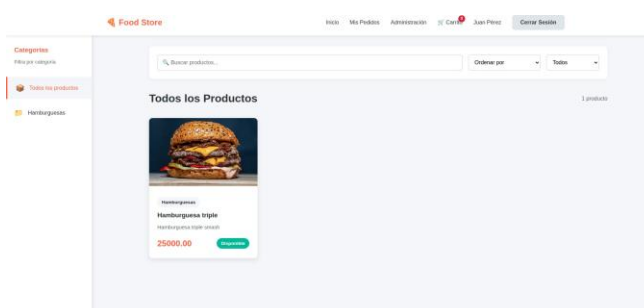
Login



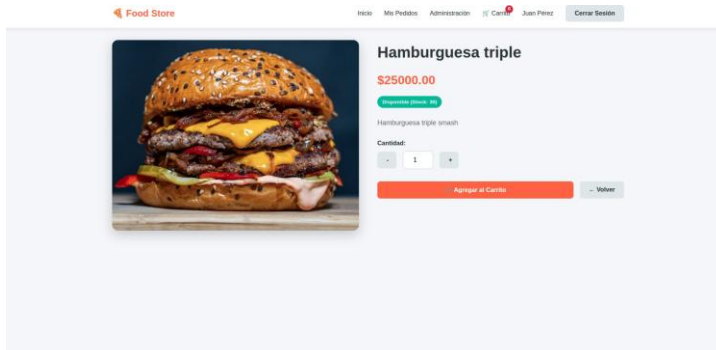
Registro



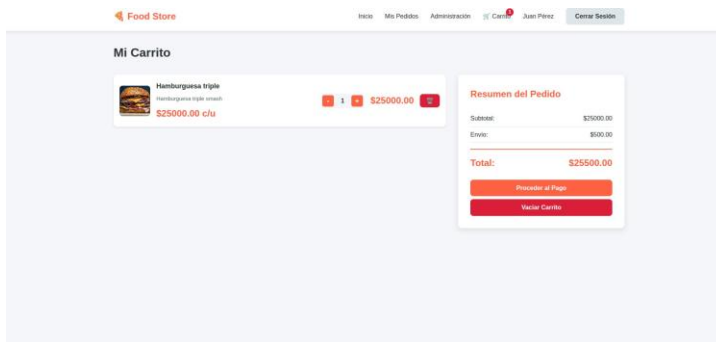
Vista Home Store



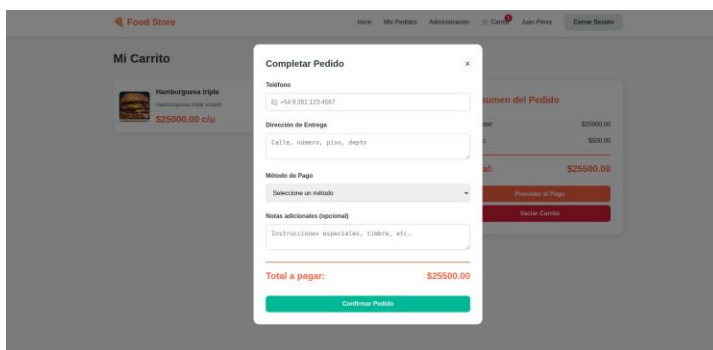
Vista Detalle de Producto



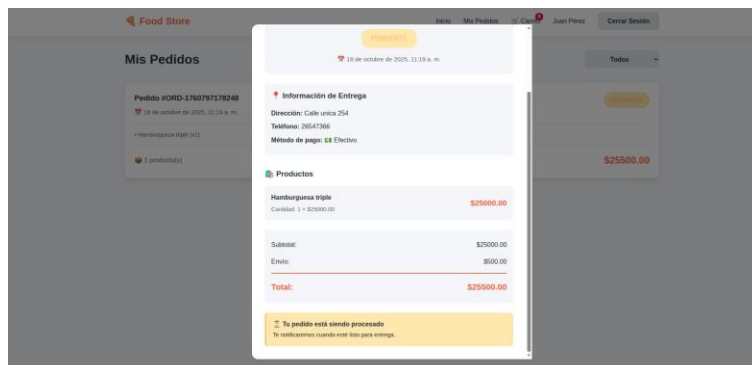
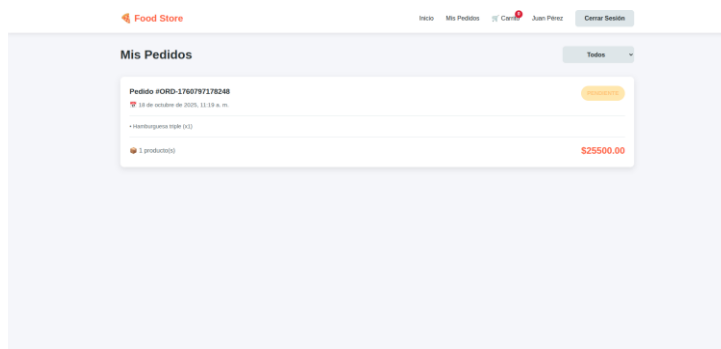
Vista de Carrito



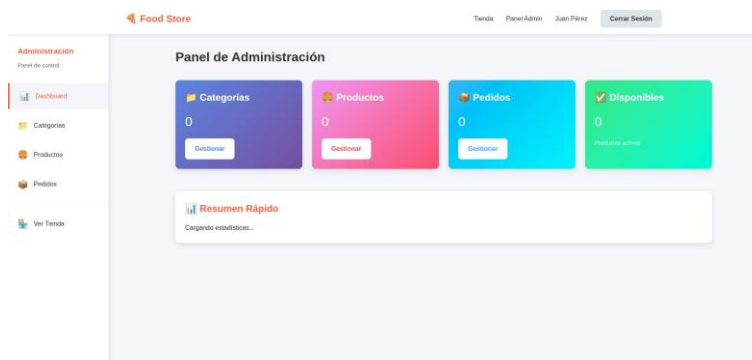
Confirmación de Pedido



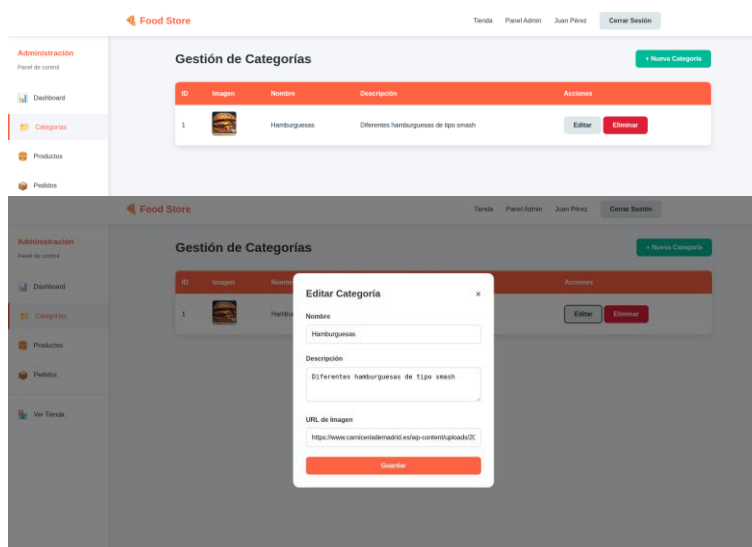
Pedidos del Cliente



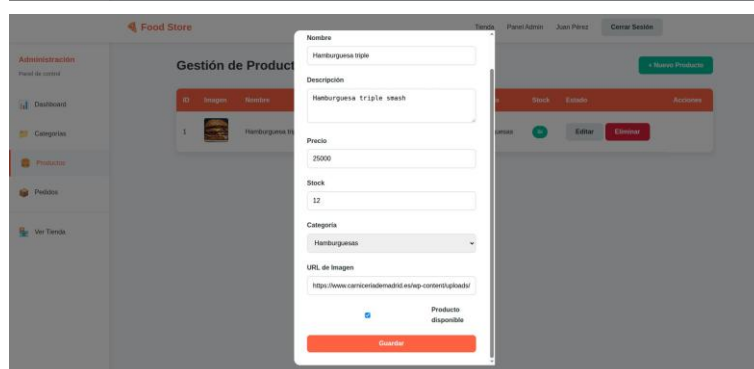
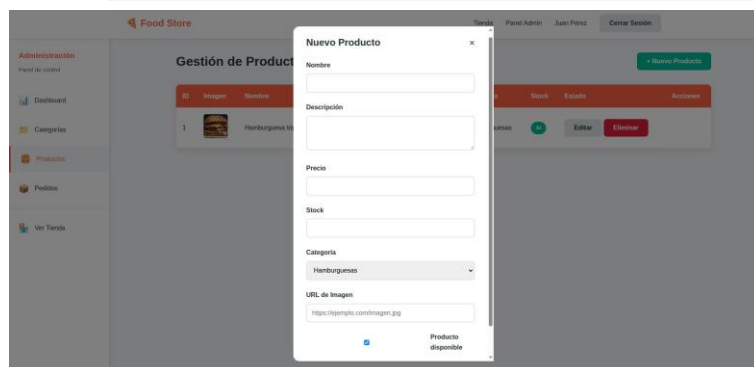
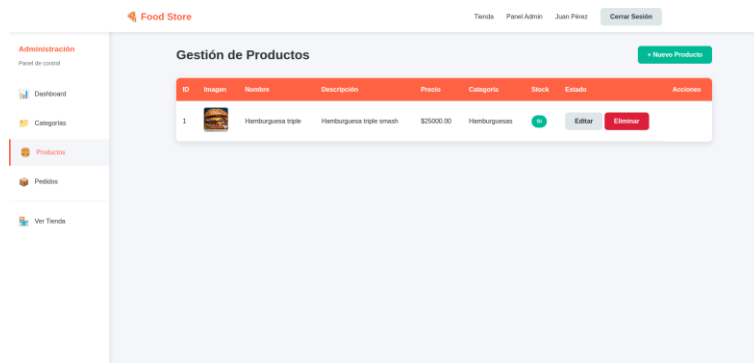
Home Admin



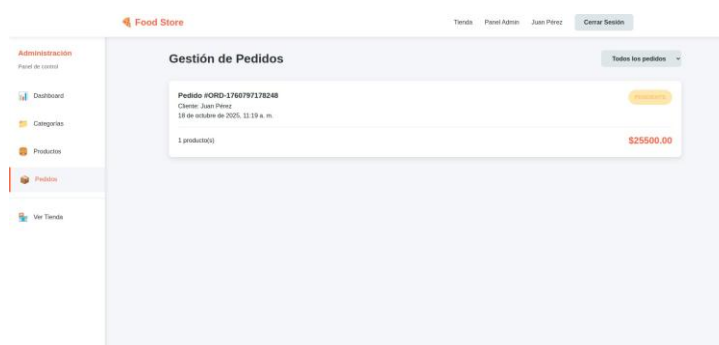
CRUD Categorías

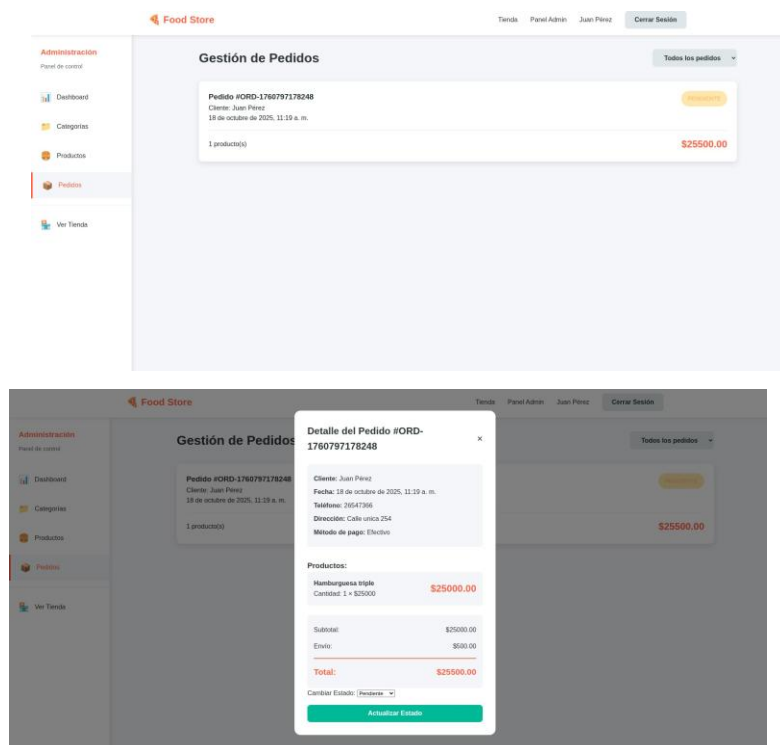


CRUD Productos



Gestión de Pedidos





F4. Autenticación y Roles

F4.1 Flujo de autenticación

La autenticación se implementa comparando las credenciales ingresadas contra los datos del archivo usuarios.json. Si el mail y la contraseña coinciden, los datos del usuario se guardan en localStorage y se redirige según el rol.

Paso	Descripción
1. Login / Registro	El usuario ingresa credenciales. El frontend hace fetch a /data/usuarios.json, busca el usuario por mail. Si existe y el campo password coincide con la contraseña ingresada, guarda el objeto usuario en localStorage (sin el password) y redirige según rol. Para el registro, en esta iteración se agrega el nuevo usuario al estado local (no se persiste en el JSON).
2. Validación de sesión	Cada página protegida verifica que exista un usuario en localStorage. Si no hay sesión, redirige al login. Valida permisos según el campo rol.
3. Cierre de sesión	Elimina el usuario de localStorage y redirige al login.

F4.2 Roles y permisos

Acción	ADMIN	USUARIO
Panel de administración	Sí	No

Get de categorías	Sí	Si
Get de productos	Sí	Si
Ver todos los pedidos	Sí	No (solo los propios)
Ver catálogo	Sí	Sí
Carrito y compras	No aplica	Sí
Ver mis pedidos	No aplica	Sí

F5. Módulos y Funcionalidades

F5.1 Módulo de Autenticación

Login (/src/pages/auth/login/)

- Formulario con campos email y contraseña.
- Validación de campos requeridos antes de enviar.
- Fetch a /data/usuarios.json para verificar credenciales.
- Manejo de errores: credenciales incorrectas o usuario inexistente.
- Redirección según el rol del usuario (admin → panel admin, usuario → home store).

Registro (/src/pages/auth/register/)

- Formulario con nombre, email y contraseña.
- Validación de campos: email con formato válido, contraseña mínimo 6 caracteres. Esta validación de longitud aplica únicamente al formulario de registro; el login no la aplica (las credenciales de prueba del README pueden tener cualquier longitud).
- Verifica que el email no esté ya registrado consultando el JSON de usuarios.
- Solo se pueden registrar clientes (rol USUARIO).
- Auto-login después del registro: guarda en localStorage y redirige al catálogo. Nota: como en esta iteración el registro no persiste el nuevo usuario en usuarios.json (solo en memoria/localStorage), si el usuario recién registrado cierra sesión no podrá volver a iniciarla hasta que el backend de la Parte 2 persista el alta. Esto es intencional para esta iteración.

F5.2 Módulo de Cliente — Catálogo

Home / Catálogo (/src/pages/store/home/)

- Fetch a /data/categorias.json para cargar el sidebar de categorías.
- Fetch a /data/productos.json para cargar el grid de productos. Solo se muestran los productos con disponible = true y eliminado = false.
- Filtrado por categoría (client-side, sobre los datos ya cargados).
- Búsqueda en tiempo real por nombre de producto.
- Ordenamiento: nombre A-Z, Z-A, precio ascendente, precio descendente.
- Cada tarjeta de producto muestra: imagen, nombre, descripción, precio y badge de disponibilidad.

A DISTANCIA

- Click en tarjeta redirige al detalle del producto.
- Badge del carrito con cantidad de ítems en el encabezado.

Detalle de Producto (/src/pages/store/productDetail/)

- Fetch a /data/productos.json, filtrar por ID del producto seleccionado.
- Muestra: imagen grande, nombre, descripción, precio, stock disponible y estado.
- Selector de cantidad con validación de stock (no permite superar el stock disponible).
- Botón Agregar al Carrito: no disponible si disponible = false o stock = 0.
- Mensaje de confirmación al agregar.
- Botón Volver al catálogo.

Carrito de Compras (/src/pages/store/cart/)

- El carrito se gestiona en localStorage (persistencia entre sesiones).
- Lista los productos agregados con: imagen, nombre, precio unitario, controles +/-, precio total por producto y botón eliminar.
- Resumen del pedido: subtotal, envío y total (total = subtotal + envío). Para esta iteración el envío es una constante fija definida en el frontend (por ejemplo ENVIO = 0 o un valor fijo declarado en un módulo de configuración); su valor debe documentarse en el README. El campo total del pedido generado refleja subtotal + envío.
- Botón Vaciar Carrito.
- Modal de checkout con campo teléfono (requerido) y selección de forma de pago. Las opciones de forma de pago deben ser exactamente TARJETA, TRANSFERENCIA y EFECTIVO, coincidiendo con el enum FormaPago del backend para que el mapeo a la API futura sea directo.
- Al confirmar: genera el objeto pedido en localStorage (en la iteración siguiente se hará POST a /api/orders).
- Validaciones: stock disponible al modificar cantidad, campos requeridos en checkout.
- Estado vacío con mensaje y botón a la tienda si el carrito está vacío.

F5.3 Módulo de Cliente — Mis Pedidos

Historial de Pedidos (/src/pages/client/orders/)

- Fetch a /data/pedidos.json, filtrar por idUsuario del usuario en sesión.
- Lista de tarjetas de pedido: número de pedido, fecha, estado con badge de color, resumen de primeros 3 productos, total.
- Click en pedido abre modal con detalle completo: estado, información de entrega, lista de productos, desglose de costos.
- Estado vacío si el usuario no tiene pedidos.

Estados de pedido y colores sugeridos:

Estado	Badge color sugerido
PENDIENTE	Amarillo / naranja

CONFIRMADO	Azul
TERMINADO	Verde
CANCELADO	Rojo

F5.4 Módulo de Administración

Dashboard (/src/pages/admin/adminHome/)

- Sidebar de navegación con enlaces a Categorías, Productos, Pedidos y Ver Tienda.
- 4 tarjetas de estadísticas: total de categorías, total de productos, total de pedidos, productos disponibles.
- Todos los conteos se obtienen con fetch a los JSON correspondientes y se calculan client-side.
- Panel de resumen: categorías activas, productos activos/inactivos, pedidos por estado.

Gestión de Categorías (/src/pages/admin/categories/)

- Fetch a /data/categorias.json para cargar la tabla.
- Tabla con columnas: ID, imagen (thumbnail), nombre, descripción, acciones (Editar / Eliminar).
- Botón Nueva Categoría abre modal con formulario: nombre (requerido), descripción (requerida), URL de imagen (requerida).
- Botón Editar carga los datos actuales en el modal para modificarlos.
- Todas las operaciones (crear, editar, eliminar) se aplican sobre el estado en memoria cargado del JSON. En la iteración siguiente se reemplazarán por POST /api/categories, PUT /api/categories/{id} y DELETE /api/categories/{id}.
- Confirmación antes de eliminar.

Gestión de Productos (/src/pages/admin/products/)

- Fetch a /data/productos.json y /data/categorias.json.
- Tabla con columnas: ID, imagen, nombre, descripción, precio, categoría, stock, estado, acciones.
- Modal para crear/editar con campos: nombre, descripción, precio (> 0), stock (>= 0), categoría (select), URL de imagen, checkbox disponible.
- Validaciones: todos los campos requeridos, precio y stock numéricos válidos, categoría debe existir en el JSON.
- Las operaciones de escritura se aplican sobre el estado en memoria. En la iteración siguiente se reemplazarán por los endpoints de la API.
- Confirmación antes de eliminar.

Gestión de Pedidos Admin (/src/pages/admin/orders/)

- Fetch a /data/pedidos.json para cargar todos los pedidos (no solo del usuario).
- Fetch a /data/usuarios.json para mostrar el nombre del cliente en cada pedido.
- Filtro por estado de pedido (client-side).
- Tarjetas de pedido: número, nombre del cliente, fecha, estado, cantidad de productos, total.

- Click en pedido abre modal con detalle completo: información del cliente, método de pago, lista de productos, desglose de costos y selector de estado.
- El cambio de estado se aplica sobre el estado en memoria. En la iteración siguiente se reemplazará por PATCH /api/orders/{id}/status.
- Ordenados por fecha, más recientes primero.

F6. Flujos de Usuario

F6.1 Flujo de compra (cliente)

1. El usuario inicia sesión (credenciales verificadas contra usuarios.json).
2. Navega el catálogo (datos de productos.json y categorías.json). Puede filtrar y buscar.
3. Hace click en un producto, ve el detalle.
4. Selecciona cantidad y hace click en Agregar al Carrito (se guarda en localStorage).
5. Accede al carrito, revisa productos y modifica cantidades si es necesario.
6. Hace click en Proceder al Pago, completa el formulario de checkout.
7. Confirma el pedido (se genera el objeto pedido en localStorage).
8. Ve mensaje de éxito y es redirigido a Mis Pedidos.

F6.2 Flujo de gestión de producto (admin)

9. El admin inicia sesión y es redirigido al Panel de Administración.
10. Va a Productos desde el sidebar.
11. Hace click en Nuevo Producto, completa el formulario (categoría del select cargado desde categorías.json).
12. Guarda el producto (se actualiza el estado en memoria).
13. Puede editar o eliminar desde la tabla.

F6.3 Flujo de gestión de pedido (admin)

14. El admin va a Pedidos desde el sidebar.
15. Ve la lista de todos los pedidos (pedidos.json). Puede filtrar por estado.
16. Hace click en un pedido para ver el detalle completo.
17. Cambia el estado desde el selector del modal.
18. Guarda el cambio (se actualiza el estado en memoria).

F7. Consideraciones Importantes

IMPORTANTE: Este proyecto NO implementa seguridad real en el frontend:

- Las contraseñas se comparan en texto plano contra el JSON (solo fines educativos).
- No hay tokens JWT.
- La validación de rol es solo frontend: localStorage es fácilmente modificable.
- Las operaciones de escritura (crear, editar, eliminar) se aplican únicamente en memoria: al recargar la página se pierde el estado modificado. Esto es intencional para esta iteración.

- SOLO para fines educativos. En la iteración siguiente se conectará al backend desarrollado en la Parte 2.

F8. Historias de Usuario — Frontend

F8.1 Autenticación

FHU-01 Login de usuario
Como: usuario del sistema
Quiero: iniciar sesión con mi email y contraseña
Para: acceder al sistema según mi rol
Prioridad: Alta
Story Points: 8

Criterios de Aceptación

1. Formulario con campos email y contraseña visibles.
2. Validación de campos requeridos antes de procesar.
3. Fetch a /data/usuarios.json para verificar credenciales.
4. Si las credenciales son incorrectas, se muestra mensaje de error.
5. Si son correctas, se guardan los datos del usuario en localStorage.
6. Se redirige al catálogo si es USUARIO, al panel admin si es ADMIN.

FHU-02 Registro de cliente
Como: visitante del sistema
Quiero: registrarme con nombre, email y contraseña
Para: poder realizar compras en el sistema
Prioridad: Alta
Story Points: 8

Criterios de Aceptación

1. Formulario con nombre, email y contraseña.
2. Validación: email con formato válido, contraseña mínimo 6 caracteres.
3. Verifica que el email no esté ya en usuarios.json.
4. Solo se registran clientes (rol USUARIO).
5. Auto-login después del registro exitoso.

F8.2 Catálogo

FHU-03 | Ver catálogo de productos

Como: cliente autenticado

Quiero: ver todos los productos disponibles con filtros y búsqueda

Para: encontrar rápidamente lo que quiero comprar

Prioridad: Alta

Story Points: 10

Criterios de Aceptación

1. Fetch a /data/productos.json carga el grid de productos.
2. Fetch a /data/categorias.json carga el sidebar de categorías.
3. El filtro por categoría actualiza el grid sin nueva petición al servidor.
4. La búsqueda por nombre funciona en tiempo real (client-side).
5. El ordenamiento (nombre, precio) funciona client-side.
6. Solo se muestran productos con disponible = true y eliminado = false.
7. Cada tarjeta tiene imagen, nombre, precio y badge de disponibilidad.

FHU-04 | Ver detalle de producto y agregar al carrito

Como: cliente autenticado

Quiero: ver la información completa de un producto y agregarlo al carrito

Para: tomar una decisión de compra informada

Prioridad: Alta

Story Points: 8

Criterios de Aceptación

1. Fetch a /data/productos.json, filtrar por ID.
2. Se muestran: imagen, nombre, descripción, precio, stock y estado.
3. Selector de cantidad respeta el stock disponible.
4. No permite agregar si disponible = false o stock = 0.
5. Al agregar, el ítem se guarda en localStorage con cantidad y precio.
6. Se muestra mensaje de confirmación.

FHU-05 | Gestionar carrito y realizar pedido

Como: cliente autenticado

Quiero: revisar mi carrito, modificar cantidades y confirmar la compra

Para: completar mi pedido con los productos seleccionados

Prioridad: Alta

Story Points: 12

Criterios de Aceptación

1. El carrito se carga desde localStorage al acceder a la página.
2. Se pueden modificar cantidades (respetando stock) y eliminar productos.
3. Se calcula subtotal por producto y total del carrito.
4. El formulario de checkout solicita teléfono y forma de pago.
5. Al confirmar, se genera el objeto pedido y se guarda en localStorage.
6. El carrito se limpia después de confirmar el pedido.
7. Estado vacío con mensaje y botón de vuelta a la tienda.

FHU-06 | Ver historial de pedidos

Como: cliente autenticado

Quiero: ver todos mis pedidos con su estado y detalle

Para: hacer seguimiento de mis compras

Prioridad: Alta

Story Points: 6

Criterios de Aceptación

1. Fetch a /data/pedidos.json, filtrar por idUsuario del usuario en sesión.
2. Lista de tarjetas con número de pedido, fecha, estado y total.
3. Click en pedido abre modal con detalle completo.
4. Badge de color según el estado del pedido.
5. Estado vacío con mensaje si no hay pedidos.

F8.3 Administración

FHU-07 | Dashboard de administración

Como: administrador autenticado

Quiero: ver un resumen del estado del sistema al ingresar al panel

Para: tener una vista rápida de categorías, productos y pedidos

Prioridad: Alta

Story Points: 6

Criterios de Aceptación

1. Accesible solo para usuarios con rol ADMIN.
2. Fetch a los 4 archivos JSON para calcular estadísticas client-side.
3. 4 tarjetas: total categorías, total productos, total pedidos, productos disponibles.
4. Panel de resumen: activos/inactivos por entidad, pedidos por estado.
5. Sidebar de navegación hacia todos los módulos de administración.

FHU-08 | CRUD de categorías (admin)

Como: administrador autenticado

Quiero: crear, editar y eliminar categorías desde la interfaz

Para: mantener organizado el catálogo de productos

Prioridad: Alta

Story Points: 10

Criterios de Aceptación

1. Fetch a /data/categorias.json para listar las categorías en tabla.
2. Modal de creación con campos nombre, descripción e imagen (todos requeridos).
3. Modal de edición carga los valores actuales del registro seleccionado.
4. Confirmación antes de eliminar.
5. Las operaciones se aplican sobre el estado en memoria.
6. La tabla se actualiza inmediatamente después de cada operación.

FHU-09 | CRUD de productos (admin)

Como: administrador autenticado

Quiero: crear, editar y eliminar productos desde la interfaz

Para: mantener el catálogo actualizado

Prioridad: Alta

Story Points: 10

Criterios de Aceptación

1. Fetch a /data/productos.json y /data/categorias.json.
2. Tabla con todos los campos del producto incluyendo nombre de categoría.
3. Modal con todos los campos: nombre, descripción, precio, stock, categoría, imagen, disponible.
4. Validaciones: precio > 0, stock >= 0, categoría existente.
5. Las operaciones se aplican sobre el estado en memoria.

FHU-10 | Gestión de pedidos (admin)

Como: administrador autenticado

Quiero: ver todos los pedidos y actualizar su estado

Para: gestionar el flujo de las órdenes de los clientes

Prioridad: Alta

Story Points: 10

Criterios de Aceptación

1. Fetch a /data/pedidos.json y /data/usuarios.json.
2. Lista todos los pedidos con nombre del cliente, fecha, estado y total.
3. Filtro por estado funciona client-side.
4. Modal de detalle muestra toda la información del pedido.
5. Select para cambiar estado del pedido (se actualiza en memoria).
6. Los pedidos se ordenan por fecha, más recientes primero.

PARTE 2

Backend JPA + Hibernate + H2 — Menú de Consola

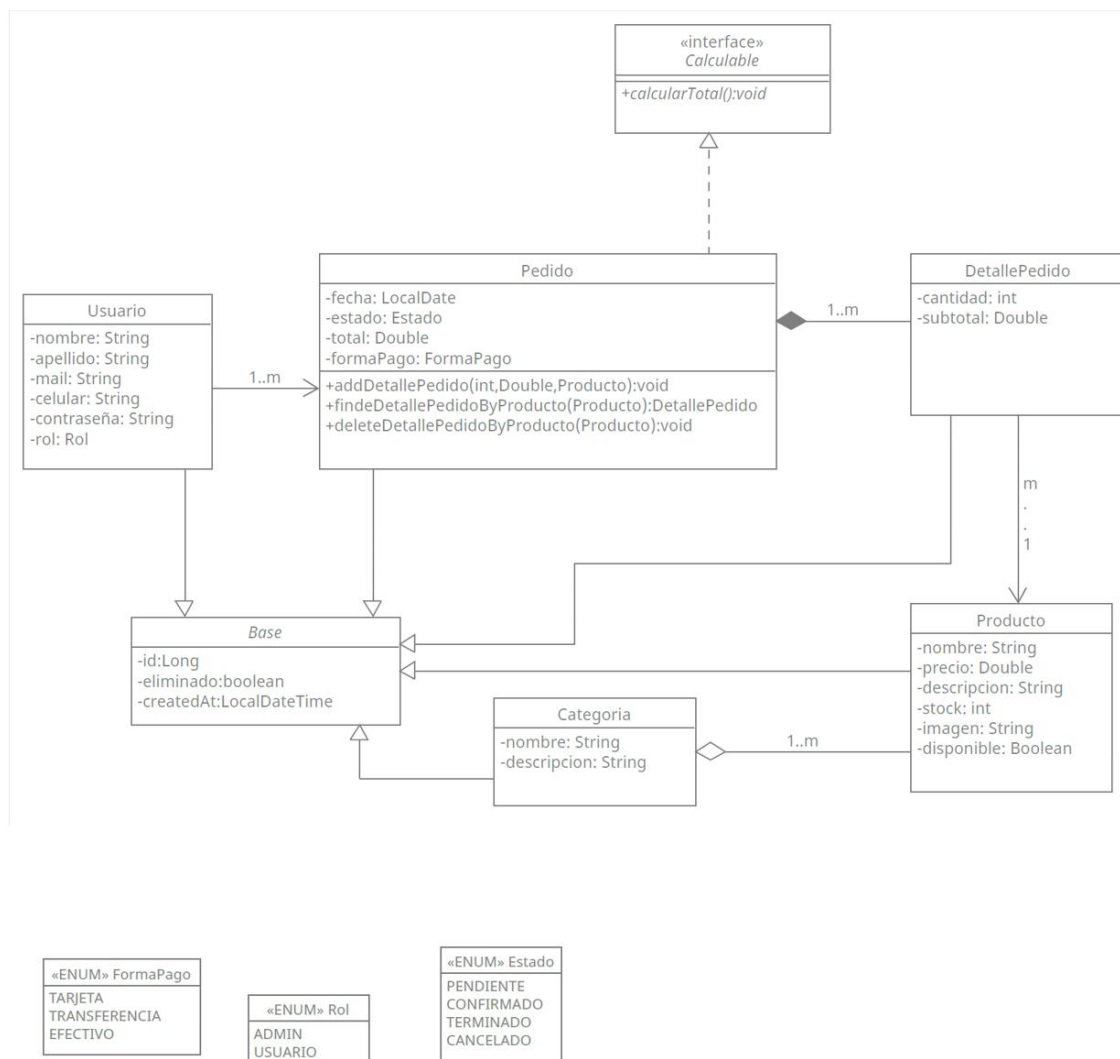
1. Objetivo General

Implementar un sistema de gestión de pedidos de comida llamado **Food Store**, utilizando Java con JPA, Hibernate y base de datos H2 en archivo. La interacción con el sistema se realiza completamente a través de un menú de consola que permite gestionar todas las entidades del modelo: Categoría, Producto, Usuario y Pedido (con sus DetallePedido).

Nota: Quienes hayan aprobado el Parcial 2 ya cuentan con la base del proyecto (BaseRepository, CategoriaRepository, ProductoRepository y el menú de Categorías y Productos). Pueden partir de esa entrega. De todas formas, esta consigna describe en detalle todos los componentes del sistema, incluyendo los ya desarrollados, para que cualquier alumno pueda construirlo desde cero.

2. Modelo de Dominio

El modelo se compone de una clase abstracta Base (anotada con `@MappedSuperclass`), una interfaz Calculable y cinco entidades JPA que extienden Base: Categoría, Producto, Usuario, Pedido y DetallePedido. A continuación se describe cada una y las relaciones entre ellas.



2.1 Entidades

Entidad	Descripción y campos propios
Base (abstracta)	Clase padre de todas las entidades. Define los campos comunes: id (Long, autogenerado con @GeneratedValue), eliminado (boolean, default false), createdAt (LocalDateTime). Anotada con @MappedSuperclass.
Calculable (interfaz)	Define el método calcularTotal():void. La clase Pedido lo implementa sumando los subtotales de sus DetallePedido y asignando el resultado a su propio campo total (no retorna valor; al ser void se invoca como pedido.calcularTotal() y luego se lee pedido.getTotal()).
Categoria	Representa una categoría de productos (ej: Hamburguesas, Bebidas). Campos: nombre (String), descripcion (String). Extiende Base.
Producto	Artículo del catálogo. Campos: nombre (String), precio (Double), descripcion (String), stock (int), imagen (String), disponible (Boolean). Extiende Base.
Usuario	Persona que opera el sistema o realiza pedidos. Campos: nombre (String), apellido (String), mail (String, único), celular (String), contrasena (String), rol (enum Rol). Extiende Base.
Pedido	Orden de compra. Campos: fecha (LocalDate), estado (enum Estado), total (Double), formaPago (enum FormaPago). Implementa Calculable. Extiende Base.
DetallePedido	Línea de un pedido. Representa un producto con su cantidad dentro de un pedido. Campos: cantidad (int), subtotal (Double), producto (@ManyToOne hacia Producto) y pedido (@ManyToOne hacia Pedido, FK del pedido al que pertenece). Extiende Base.

2.2 Relaciones entre entidades

Las relaciones hacia Categoria, Producto y Usuario son unidireccionales; la relación Pedido–DetallePedido es bidireccional (Pedido mantiene la colección con mappedBy y cada DetallePedido conoce a su Pedido). La siguiente tabla describe cada una con su tipo UML, cardinalidad, anotación JPA y comportamiento:

Tipo UML	Relación	Cardinalidad	Descripción y anotación JPA
Agregación unidireccional	Categoria → Producto	1 Categoria tiene muchos Productos	Producto conoce a su Categoria; Categoria no tiene lista de Productos. Producto declara: @ManyToOne @JoinColumn(name="categoria_id") private Categoria categoria. La categoría puede existir sin productos.
Composición bidireccional	Pedido → DetallePedido	1 Pedido tiene muchos DetallePedido	Relación bidireccional: los DetallePedido son creados y gestionados exclusivamente por Pedido a través del método addDetallePedido(int cantidad, Producto producto), y cada DetallePedido conoce a su Pedido

			mediante la FK pedido_id. Pedido declara: <pre>@OneToMany(mappedBy="pedido", cascade=CascadeType.ALL, orphanRemoval=true) private List<DetallePedido> detalles = new ArrayList<>(). El valor de mappedBy ("pedido") debe coincidir exactamente con el nombre del campo @ManyToOne Pedido pedido declarado en DetallePedido. El cascade y orphanRemoval solo tienen efecto si el Pedido se eliminara físicamente; como en este sistema las bajas son siempre lógicas (eliminado = true), los DetallePedido permanecen en la BD.</pre>
Asociación unidireccional	DetallePedido → Producto	Muchos DetallePedido referencian 1 Producto	DetallePedido conoce al Producto que representa; Producto no conoce sus detalles. DetallePedido declara: <pre>@ManyToOne @JoinColumn(name="producto_id") private Producto producto</pre>
Asociación unidireccional	Usuario → Pedido	1 Usuario puede tener muchos Pedidos	Usuario conoce a su Pedido; Usuario tiene lista de Pedidos. Usuario declara: <pre>@OneToMany(cascade = CascadeType.ALL, fetch = FetchType.LAZY) @JoinColumn(name = "usuario_id") @Builder.Default private Set<Pedido> pedidos = new HashSet<>(). Para obtener los pedidos de un usuario se usa la consulta JPQL en PedidoRepository (buscarPorUsuario).</pre>

Nota sobre addDetallePedido(int cantidad, Producto producto): el método addDetallePedido(int cantidad, Producto producto) de la clase Pedido crea una instancia de DetallePedido, calcula su subtotal (producto.getPrecio() * cantidad), establece la referencia bidireccional detalle.setPedido(this), y agrega el detalle a la colección this.detalles. Es el único punto de creación de DetallePedido en el sistema. La colección detalles debe inicializarse en su declaración (= new ArrayList<>()) para evitar NullPointerException al invocar este método sobre un Pedido recién instanciado.

2.3 Enumerados

Enum	Valores
Rol	ADMIN, USUARIO
Estado	PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO
FormaPago	TARJETA, TRANSFERENCIA, EFECTIVO

3. Estructura del Proyecto

El proyecto es Gradle con Java. Respetar la siguiente estructura de paquetes dentro de `src/main/java/`:

```
com.tp.jpa/
  model/                <- entidades JPA (Base, Categoria, Producto,
                        Usuario, Pedido, DetallePedido)
  model/enums/          <- enumerados (Rol, Estado, FormaPago)
  util/                 <- JPAUtil.java (fábrica de
                        EntityManagerFactory)
  repository/           <- repositorios (BaseRepository, y uno por
                        entidad)
  Main.java             <- clase principal con el menú de consola
```

La configuración de la base de datos se encuentra en `src/main/resources/META-INF/persistence.xml`. Se usa H2 en modo archivo (`jdbc:h2:file:./data/jpa_db`). Hibernate gestiona el esquema automáticamente con `hbm2ddl.auto = update`.

4. Componentes del Sistema

4.1 JPAUtil

Clase utilitaria que mantiene una única instancia de `EntityManagerFactory` para toda la aplicación. Se obtiene llamando a `JPAUtil.getEntityManagerFactory()`. El factory se cierra al finalizar la aplicación con `JPAUtil.close()`.

4.2 BaseRepository<T>

Clase abstracta genérica que recibe la `Class<T>` de la entidad por constructor y obtiene el `EntityManagerFactory` desde `JPAUtil`. Implementa las operaciones CRUD comunes para todas las entidades. Cada método abre su propio `EntityManager` al inicio y lo cierra en un bloque `finally`.

Métodos que debe implementar:

Método	Comportamiento esperado
<code>guardar(T entity): T</code>	Abre <code>EntityManager</code> , inicia transacción y persiste la entidad: si su id es null usa <code>persist()</code> (alta, el ID lo asigna la BD), si ya tiene id usa <code>merge()</code> (actualización). Hace <code>commit</code> y retorna la entidad gestionada. En caso de excepción hace <code>rollback</code> . Cierra <code>EntityManager</code> en <code>finally</code> . IMPORTANTE: para mostrar el ID generado en un alta debe leerse del objeto retornado por <code>guardar()</code> (con <code>persist()</code> la propia instancia recibe el ID; si se optara por <code>merge()</code> el ID solo está en la copia devuelta, nunca en el objeto original).
<code>buscarPorId(Long id): Optional<T></code>	Abre <code>EntityManager</code> , busca con <code>find(clase, id)</code> . Si existe retorna <code>Optional.of(entidad)</code> , si no existe retorna <code>Optional.empty()</code> . Cierra <code>EntityManager</code> en <code>finally</code> .

listarActivos(): List<T>	Abre EntityManager, ejecuta JPQL: SELECT e FROM NombreEntidad e WHERE e.eliminado = false. Retorna List<T>. Cierra EntityManager en finally.
eliminarLogico(Long id): boolean	Abre EntityManager, busca la entidad por ID. Si existe, inicia transacción, establece eliminado = true, sincroniza con merge() (la entidad ya tiene id, por lo que merge() es correcto), hace commit y retorna true. Si no existe retorna false. Cierra EntityManager en finally.

Nota técnica: la consulta JPQL de listarActivos() debe construirse usando el nombre simple de la clase (getEntityClass().getSimpleName()) para que funcione correctamente con todas las entidades que extienden Base.

4.3 CategoriaRepository

Extiende BaseRepository<Categoria>. El constructor llama a super(Categoria.class). No requiere métodos adicionales: hereda todo el CRUD del repositorio base.

4.4 ProductoRepository

Extiende BaseRepository<Producto>. El constructor llama a super(Producto.class). Además del CRUD heredado implementa:

buscarPorCategoria(Long categoriaId): List<Producto>

Retorna los productos activos que pertenecen a la categoría indicada. Debe usar JPQL con parámetro nombrado, filtrar por eliminado = false, y retornar un List<Producto> sin casteos manuales. Incluir comentario explicando la consulta.

```
// Consulta JPQL: retorna productos activos de una categoría específica
// Filtra por categoria.id y por eliminado = false para excluir bajas lógicas
String jpql = "SELECT p FROM Producto p WHERE p.categoria.id = :catId AND p.eliminado = false";
List<Producto> q = em.createQuery(jpql, Producto.class)
    .setParameter("catId", categoriaId)
    .getResultList();
```

4.5 UsuarioRepository

Extiende BaseRepository<Usuario>. El constructor llama a super(Usuario.class). Además del CRUD heredado implementa:

buscarPorMail(String mail): Optional<Usuario>

Retorna el usuario activo con el mail indicado. Usar JPQL con parámetro nombrado, filtrar por eliminado = false. Retornar Optional.of(usuario) si existe, Optional.empty() si no. Incluir comentario explicando la consulta.

```
// Consulta JPQL: busca un usuario activo por su dirección de correo electrónico
```

```
// Retorna Optional para manejar el caso en que el mail no esté
registrado
String jpql = "SELECT u FROM Usuario u WHERE u.mail = :mail AND
u.eliminado = false";
TypedQuery<Usuario> q = em.createQuery(jpql, Usuario.class);
q.setParameter("mail", mail);
List<Usuario> res = q.getResultList();
return res.isEmpty() ? Optional.empty() : Optional.of(res.get(0));
```

4.6 PedidoRepository

Extiende BaseRepository<Pedido>. El constructor llama a super(Pedido.class). Además del CRUD heredado implementa:

buscarPorUsuario(Long idUsuario): List<Pedido>

Retorna los pedidos activos del usuario indicado. Usar JPQL con parámetro nombrado, filtrar por eliminado = false.

```
// Consulta JPQL: retorna todos los pedidos activos de un usuario dado
su ID
// Filtra por eliminado = false para excluir pedidos dados de baja
lógica
String jpql = "SELECT p FROM Pedido p WHERE p.usuario.id = :uid AND
p.eliminado = false";
List<Pedido> q = em.createQuery(jpql, Pedido.class)
.setParameter("uid", idUsuario)
.getResultList();
```

buscarPorEstado(Estado estado): List<Pedido>

Retorna los pedidos activos que coincidan con el estado indicado. Usar JPQL con parámetro nombrado, filtrar por eliminado = false.

```
// Consulta JPQL: retorna todos los pedidos activos con un estado
específico
// Útil para filtrar PENDIENTE, CONFIRMADO, TERMINADO o CANCELADO
String jpql = "SELECT p FROM Pedido p WHERE p.estado = :estado AND
p.eliminado = false";
List<Pedido> q = em.createQuery(jpql, Pedido.class)
.setParameter("estado", estado)
.getResultList();
```

5. Menú de Consola

La clase Main presenta un menú principal con las siguientes secciones. El orden de uso natural es: primero crear Categorías, luego Productos (asociándolos a una categoría), luego Usuarios, y por último Pedidos.

Opción del menú principal	Descripción
1. Gestionar Categorías	ABM completo de categorías.
2. Gestionar Productos	ABM completo de productos, asociados a una categoría.
3. Gestionar Usuarios	ABM completo de usuarios con búsqueda por mail.
4. Gestionar Pedidos	Alta de pedido con detalles, cambio de estado, baja y listados.
5. Reportes	Consultas JPQL: productos por categoría, pedidos por usuario, pedidos por estado, total facturado.
0. Salir	Cierra el EntityManagerFactory y termina la aplicación.

5.1 Submenú Categorías

Este submenú es el punto de entrada obligatorio del sistema. Antes de crear productos es necesario tener al menos una categoría activa.

Opción	Comportamiento
1. Alta	Solicitar nombre (obligatorio, no puede estar vacío) y descripción (opcional). Crear instancia de Categoría, llamar a categoriaRepo.guardar(). Mostrar el ID generado por la BD.
2. Modificar	Listar categorías activas con listarActivos(). Solicitar ID. Si no corresponde a una categoría activa, mostrar error y volver al menú. Mostrar valores actuales. Solicitar nuevo nombre y descripción: si el usuario deja el campo vacío (presiona Enter sin escribir), conservar el valor anterior. Llamar a guardar() con la entidad modificada.
3. Baja lógica	Solicitar ID. Llamar a eliminarLogico(id). Si retorna false, mostrar error (no encontrado o ya dado de baja). Si retorna true, confirmar mostrando el nombre de la categoría afectada.
4. Listado	Llamar a listarActivos() y mostrar todas las categorías activas con: ID, nombre y descripción.
0. Volver	Retornar al menú principal.

5.2 Submenú Productos

Los productos deben estar asociados a una categoría. Si no hay categorías activas al dar de alta un producto, se informa y se cancela la operación.

Opción	Comportamiento
1. Alta	Listar categorías activas. Si no hay ninguna, mostrar error y volver. Solicitar selección de categoría por ID. Solicitar: nombre (obligatorio), descripción, precio (Double, mayor a 0), stock (int, mayor o igual a 0), imagen (opcional), disponible (S/N, default true). Validar precio y stock: si son inválidos, mostrar error y no persistir. Crear instancia de Producto con la Categoría seleccionada. Llamar a productoRepo.guardar(). Mostrar ID generado y categoría asignada.
2. Modificar	Listar productos activos con listarActivos(). Solicitar ID. Si no existe o está dado de baja, mostrar error. Mostrar valores actuales. Solicitar nuevos valores para nombre, precio y

	stock: campo vacío conserva valor anterior. Validar precio > 0 y stock >= 0 si se ingresa un nuevo valor. Guardar cambios.
3. Baja lógica	Solicitar ID. Llamar a eliminarLogico(id). Si retorna false, mostrar error. Si retorna true, confirmar mostrando el nombre del producto afectado.
4. Listado	Llamar a listarActivos() y mostrar: ID, nombre, precio, stock, estado de disponibilidad y nombre de su categoría.
0. Volver	Retornar al menú principal.

5.3 Submenú Usuarios

Los usuarios son necesarios para generar pedidos. Se recomienda crear al menos un usuario antes de acceder al menú de Pedidos.

Opción	Comportamiento
1. Alta	Solicitar: nombre, apellido, mail, celular (opcional), contraseña y rol (mostrar opciones: ADMIN / USUARIO). Validar que el mail no esté ya en uso llamando a usuarioRepo.buscarPorMail(mail): si existe usuario activo con ese mail, mostrar error y no persistir. Crear instancia de Usuario. Llamar a usuarioRepo.guardar(). Mostrar ID generado.
2. Modificar	Listar usuarios activos. Solicitar ID. Si no existe o está dado de baja, mostrar error. Mostrar valores actuales. Solicitar nuevos valores para nombre, apellido, celular y contraseña: campo vacío conserva valor anterior. Si se cambia el mail, validar que no esté en uso por otro usuario (buscarPorMail y verificar que el ID sea distinto). Guardar cambios.
3. Baja lógica	Solicitar ID. Llamar a eliminarLogico(id). Si retorna false, mostrar error. Si retorna true, confirmar mostrando nombre y apellido del usuario. Sus pedidos permanecen en el sistema.
4. Listado	Llamar a listarActivos() y mostrar: ID, nombre completo, mail y rol.
5. Buscar por mail	Solicitar mail. Llamar a usuarioRepo.buscarPorMail(mail). Si el Optional contiene un usuario, mostrar todos sus datos (sin mostrar la contraseña). Si está vacío, informar que no existe usuario activo con ese mail.
0. Volver	Retornar al menú principal.

5.4 Submenú Pedidos

El alta de pedido es la operación más compleja del sistema. Involucra validaciones de stock, cálculo de subtotales, reducción de inventario y debe ejecutarse dentro de una única transacción atómica.

Opción	Comportamiento
1. Alta de pedido	Ver detalle completo en sección 5.4.1.
2. Cambiar estado	Solicitar ID de pedido. Si no existe o está dado de baja, mostrar error. Mostrar estado actual. Mostrar opciones: PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO. Actualizar campo estado en la entidad. Llamar a pedidoRepo.guardar(). Confirmar mostrando ID y nuevo estado.

3. Baja lógica	Solicitar ID. Llamar a eliminarLogico(id). Si retorna false, mostrar error. Si retorna true, confirmar mostrando ID y total del pedido. El stock de los productos NO se restaura. Los DetallePedido permanecen en la BD.
4. Listado	Llamar a listarActivos() y mostrar para cada pedido: ID, fecha, estado, forma de pago, nombre del usuario y total.
5. Pedidos por usuario	Listar usuarios activos, solicitar selección. Llamar a pedidoRepo.buscarPorUsuario(idUsuario). Mostrar: ID, fecha, estado y total de cada pedido. Si la lista está vacía, informarlo.
6. Pedidos por estado	Mostrar los estados disponibles, solicitar selección. Llamar a pedidoRepo.buscarPorEstado(estado). Mostrar: ID, fecha, nombre de usuario y total. Si la lista está vacía, informarlo.
0. Volver	Retornar al menú principal.

5.4.1 Alta de Pedido — Flujo detallado

Este flujo debe ejecutarse dentro de una única transacción. Si falla cualquier validación, se hace rollback completo y no se modifica nada en la base de datos.

- Listar usuarios activos y solicitar al operador que seleccione uno por ID. Si no existen usuarios activos, informar y cancelar.
- Solicitar la forma de pago mostrando las opciones del enum FormaPago: TARJETA, TRANSFERENCIA, EFECTIVO.
- Ingresar productos al pedido (ciclo repetible):
 - Mostrar el catálogo de productos activos con ID, nombre, precio y stock disponible.
 - Solicitar ID del producto a agregar. Si no existe o está dado de baja, mostrar error.
 - Verificar que el producto tenga disponible = true. Si no, mostrar error y no agregar.
 - Solicitar cantidad (entero mayor a 0). Verificar que el stock del producto sea suficiente. Si no alcanza, mostrar el stock disponible e informar el error.
 - Si las validaciones pasan, agregar el producto y la cantidad a una lista temporal en memoria (aún no se persiste nada).
 - Preguntar si desea agregar otro producto. Repetir hasta que el operador indique que no.
- Si la lista de productos está vacía al confirmar, informar que el pedido debe tener al menos un detalle y cancelar.
- Una vez confirmada la lista, abrir un único EntityManager e iniciar una sola transacción (todo el alta ocurre con este mismo EntityManager; la lista temporal previa solo guarda el ID del producto y la cantidad, no entidades de otro EntityManager):
 - Crear la instancia de Pedido con el usuario, fecha actual (LocalDate.now()), estado PENDIENTE, forma de pago seleccionada.
 - Por cada producto en la lista temporal: recuperar el Producto gestionado con em.find(Producto.class, idProducto) dentro de este EntityManager, crear un DetallePedido con la cantidad, calcular subtotal = producto.getPrecio() * cantidad, y asociarlo al Pedido mediante pedido.addDetallePedido(...).
 - Llamar a pedido.calcularTotal() (suma de subtotales de todos los detalles).
 - Reducir el stock de cada producto: producto.setStock(producto.getStock() - cantidad). Como el Producto fue recuperado con em.find() en este mismo EntityManager está gestionado, por lo que el cambio se sincroniza automáticamente al hacer commit (no hace falta merge() explícito).

- Persistir el Pedido nuevo con `em.persist(pedido)`; por el `cascade = CascadeType.ALL` los `DetallePedido` asociados se persisten en la misma operación.
- Hacer `commit`.
- Si ocurre cualquier excepción durante la transacción, hacer `rollback` completo. Mostrar mensaje de error.
- Al completar exitosamente, mostrar: ID generado, fecha, usuario, forma de pago, listado de productos con cantidades y subtotales, y el total del pedido.

5.5 Submenú Reportes

Opción	Comportamiento
1. Productos por categoría	Listar categorías activas y solicitar selección. Llamar a <code>productoRepo.buscarPorCategoria(idCategoria)</code> . Mostrar: ID, nombre, precio y stock de cada producto. Si no hay productos activos en esa categoría, informarlo explícitamente.
2. Pedidos por usuario	Listar usuarios activos y solicitar selección. Llamar a <code>pedidoRepo.buscarPorUsuario(idUsuario)</code> . Mostrar: ID, fecha, estado, forma de pago y total de cada pedido. Si no hay pedidos, informarlo.
3. Pedidos por estado	Mostrar opciones del enum <code>Estado</code> y solicitar selección. Llamar a <code>pedidoRepo.buscarPorEstado(estado)</code> . Mostrar: ID, fecha, nombre de usuario y total. Si no hay pedidos con ese estado, informarlo.
4. Total facturado	Obtener los pedidos con estado <code>TERMINADO</code> llamando a <code>pedidoRepo.buscarPorEstado(Estado.TERMINADO)</code> . Sumar los campos total de todos los pedidos del resultado (por ejemplo con <code>mapToDouble(Pedido::getTotal).sum()</code> , considerando null como 0). Mostrar el resultado formateado a dos decimales con <code>String.format(Locale.US, "%.2f", total)</code> para evitar el error de representación propio de <code>Double</code> (ej: Total facturado: \$12500.00). Si no hay pedidos terminados, mostrar \$0.00.
0. Volver	Retornar al menú principal.

6. Reglas Técnicas

- Cada método de repositorio debe abrir su propio `EntityManager` al inicio y cerrarlo en un bloque `finally`, independientemente de si la operación fue exitosa o no.
- Las operaciones de escritura (`guardar`, `eliminarLogico`) deben manejar la transacción con `begin()` / `commit()` y `rollback()` en caso de excepción.
- El alta de pedido debe ejecutarse en una única transacción: si cualquier validación falla después de iniciada, se hace `rollback` completo y no se modifica ningún dato en la BD.
- Los métodos con JPQL personalizado deben incluir un comentario que explique qué hace la consulta.
- Las bajas son siempre lógicas: `eliminado = true`. El registro permanece en la BD y no debe aparecer en ningún listado activo.
- Dejar un campo en blanco durante una modificación conserva el valor anterior (no se pisa con null ni con cadena vacía).

- Al finalizar la aplicación (opción 0 del menú principal), llamar a `JPAUtil.close()` para cerrar el `EntityManagerFactory` correctamente.

7. Historias de Usuario — Backend

7.1 Repositorios

HU-01 | BaseRepository con CRUD genérico

Como: desarrollador

Quiero: contar con un `BaseRepository<T>` que implemente las operaciones CRUD comunes

Para: no repetir código de persistencia en cada repositorio específico

Prioridad: Alta

Story Points: 18

Criterios de Aceptación

1. `guardar()` abre su propia transacción; usa `persist()` si el id es null (alta) o `merge()` si ya tiene id (actualización), y hace `commit`. Hace `rollback` ante excepción. Retorna la entidad gestionada, de la cual debe leerse el ID generado.
2. `buscarPorId()` retorna `Optional<T>`: `Optional.of(entidad)` si existe, `Optional.empty()` si no.
3. `listarActivos()` usa JPQL con `WHERE e.eliminado = false` y retorna `List<T>`.
4. `eliminarLogico()` busca por ID, establece `eliminado = true`, persiste y retorna `true`. Retorna `false` si no encuentra el registro.
5. Cada método cierra el `EntityManager` en un bloque `finally`.

HU-02 | CategoriaRepository y ProductoRepository

Como: desarrollador

Quiero: contar con `CategoriaRepository` y `ProductoRepository` que extiendan `BaseRepository`

Para: operar sobre cada entidad sin reescribir el CRUD base

Prioridad: Alta

Story Points: 12

Criterios de Aceptación

1. `CategoriaRepository` extiende `BaseRepository<Categoria>` y llama a `super(Categoria.class)`.
2. `ProductoRepository` extiende `BaseRepository<Producto>` y llama a `super(Producto.class)`.
3. `ProductoRepository` incluye `buscarPorCategoria(Long categoriaId)` con JPQL tipado.
4. La consulta JPQL filtra por `categoria.id = :catId` y `eliminado = false`.
5. El método retorna `List<Producto>`.

6. El método incluye un comentario explicando la consulta JPQL.

HU-03 | UsuarioRepository con buscarPorMail

Como: desarrollador

Quiero: contar con UsuarioRepository que extienda BaseRepository<Usuario> y provea buscarPorMail()

Para: validar unicidad de mail y buscar usuarios sin reescribir el CRUD base

Prioridad: Alta

Story Points: 8

Criterios de Aceptación

1. UsuarioRepository extiende BaseRepository<Usuario> y llama a super(Usuario.class).
2. buscarPorMail() usa JPQL con parámetro nombrado :mail y WHERE e.eliminado = false.
3. Retorna Optional<Usuario>: Optional.of(usuario) si existe, Optional.empty() si no.
4. La consulta se ejecuta con TypedQuery<Usuario> y devuelve el resultado mediante getResultList().
5. El método cierra el EntityManager en un bloque finally.
6. Incluye un comentario explicando la consulta JPQL.

HU-04 | PedidoRepository con consultas JPQL

Como: desarrollador

Quiero: contar con PedidoRepository con buscarPorUsuario() y buscarPorEstado()

Para: obtener pedidos filtrados sin escribir JPQL fuera del repositorio

Prioridad: Alta

Story Points: 10

Criterios de Aceptación

1. PedidoRepository extiende BaseRepository<Pedido> y llama a super(Pedido.class).
2. buscarPorUsuario() filtra por usuario.id = :uid y eliminado = false. Retorna List<Pedido>.
3. buscarPorEstado() filtra por estado = :estado y eliminado = false. Retorna List<Pedido>.
4. Ambos usan List<Pedido>.
5. Cada método cierra el EntityManager en un bloque finally.
6. Cada método incluye un comentario explicando la consulta JPQL.

7.2 Categorías

HU-05 | Dar de alta una categoría

Como: operador del sistema

Quiero: crear una nueva categoría ingresando nombre y descripción

Para: organizar los productos del catálogo en grupos temáticos

Prioridad: Alta

Story Points: 8

Criterios de Aceptación

1. El sistema solicita nombre y descripción por consola.
2. Si el nombre está vacío, el sistema informa el error y no persiste.
3. Al guardar exitosamente, se muestra el ID generado por la base de datos.
4. La categoría queda con eliminado = false.

HU-06 | Modificar una categoría existente

Como: operador del sistema

Quiero: editar el nombre o la descripción de una categoría ya creada

Para: corregir errores sin tener que borrar y recrear el registro

Prioridad: Alta

Story Points: 10

Criterios de Aceptación

1. El sistema lista las categorías activas antes de pedir el ID.
2. Si el ID no corresponde a ninguna categoría activa, se muestra un mensaje de error.
3. Se muestran los valores actuales antes de pedir los nuevos.
4. Dejar un campo en blanco mantiene el valor anterior.
5. El cambio se persiste correctamente en la base de datos.

HU-07 | Dar de baja una categoría

Como: operador del sistema

Quiero: dar de baja una categoría que ya no se utiliza

Para: ocultarla del sistema sin perder el historial de datos

Prioridad: Alta

Story Points: 7

Criterios de Aceptación

1. La baja es lógica: se establece eliminado = true, el registro permanece en la BD.

2. Si el ID no existe o ya está dado de baja, el sistema informa el error.
3. La categoría dada de baja no aparece en ningún listado activo.
4. Se confirma la operación mostrando el nombre de la categoría afectada.

7.3 Productos

HU-08 | Dar de alta un producto

Como: operador del sistema

Quiero: registrar un nuevo producto asociándolo a una categoría existente

Para: incorporar artículos al catálogo con toda su información básica

Prioridad: Alta

Story Points: 12

Criterios de Aceptación

1. El sistema lista las categorías activas para que el operador seleccione una.
2. Si no hay categorías activas, se informa y se cancela la operación.
3. Se solicitan: nombre (obligatorio), descripción, precio (mayor a 0), stock (mayor o igual a 0), imagen y disponible.
4. Si precio o stock tienen valores inválidos, se informa el error y no se persiste.
5. Al guardar se muestra el ID generado y la categoría asignada.

HU-09 | Modificar un producto

Como: operador del sistema

Quiero: actualizar el nombre, precio y stock de un producto existente

Para: mantener el catálogo actualizado sin recrear el registro

Prioridad: Alta

Story Points: 10

Criterios de Aceptación

1. El sistema lista los productos activos antes de pedir el ID.
2. Si el ID no existe o el producto está dado de baja, se muestra error.
3. Se muestran los valores actuales antes de pedir los nuevos.
4. Dejar un campo en blanco conserva el valor anterior.
5. Precio no puede actualizarse a un valor menor o igual a 0.
6. Stock no puede actualizarse a un valor negativo.

HU-10 | Dar de baja un producto

Como: operador del sistema

Quiero: dar de baja un producto que ya no está disponible

Para: retirarlo del catálogo activo sin eliminar su historial

Prioridad: Alta

Story Points: 8

Criterios de Aceptación

1. La baja es lógica: eliminado = true, el registro permanece en la BD.
2. Si el ID no existe o ya está dado de baja, se informa el error.
3. El producto dado de baja no aparece en el listado de productos activos.
4. Se muestra confirmación con el nombre del producto afectado.

HU-11 | Consulta JPQL: Productos por categoría

Como: operador del sistema

Quiero: ver todos los productos activos que pertenecen a una categoría específica

Para: consultar el catálogo filtrado sin revisar todos los productos

Prioridad: Alta

Story Points: 10

Criterios de Aceptación

1. El sistema lista las categorías activas para que el operador seleccione una.
2. La consulta está implementada en ProductoRepository con JPQL y parámetro nombrado :catId.
3. El método retorna List<Producto> sin casteos manuales.
4. Solo se incluyen productos con eliminado = false.
5. El resultado muestra: ID, nombre, precio y stock de cada producto.
6. Si la categoría no tiene productos activos, se informa explícitamente.

7.4 Usuarios

HU-12 | Dar de alta un usuario

Como: operador del sistema

Quiero: registrar un nuevo usuario con sus datos y rol

Para: que pueda ser asociado a pedidos en el sistema

Prioridad: Alta

Criterios de Aceptación

1. Se solicitan: nombre, apellido, mail, celular, contraseña y rol (ADMIN / USUARIO).
2. Si el mail ya está en uso (buscarPorMail retorna un Optional no vacío), se informa el error y no se persiste.
3. Al guardar exitosamente se muestra el ID generado.
4. El usuario queda con eliminado = false.

HU-13 | Modificar un usuario

Como: operador del sistema

Quiero: editar los datos de un usuario existente

Para: mantener la información actualizada

Prioridad: Alta

Story Points: 8

Criterios de Aceptación

1. El sistema lista los usuarios activos antes de pedir el ID.
2. Si el ID no corresponde a un usuario activo, se muestra error.
3. Se muestran los valores actuales antes de pedir los nuevos.
4. Dejar un campo en blanco conserva el valor anterior.
5. Si se modifica el mail, verificar que no esté en uso por otro usuario activo.
6. El cambio se persiste correctamente.

HU-14 | Dar de baja un usuario

Como: operador del sistema

Quiero: dar de baja un usuario que ya no utiliza el sistema

Para: ocultarlo sin perder el historial de pedidos

Prioridad: Alta

Story Points: 6

Criterios de Aceptación

1. La baja es lógica: eliminado = true, el registro permanece en la BD.
2. Si el ID no existe o ya está dado de baja, se informa el error.
3. El usuario dado de baja no aparece en ningún listado activo.

4. Se confirma mostrando el nombre completo del usuario afectado.
5. Sus pedidos permanecen en el sistema sin modificación.

HU-15 | Buscar usuario por mail

Como: operador del sistema

Quiero: buscar un usuario ingresando su dirección de mail

Para: consultar rápidamente sus datos sin recorrer el listado

Prioridad: Alta

Story Points: 6

Criterios de Aceptación

1. Se solicita el mail por consola.
2. Se llama a `usuarioRepo.buscarPorMail(mail)`.
3. Si el Optional no está vacío, se muestran todos los datos del usuario (sin mostrar la contraseña).
4. Si el Optional está vacío, se informa que no existe usuario activo con ese mail.

7.5 Pedidos

HU-16 | Dar de alta un pedido con detalles

Como: operador del sistema

Quiero: registrar un pedido completo con sus productos y cantidades

Para: registrar una compra y actualizar el inventario automáticamente

Prioridad: Alta

Story Points: 20

Criterios de Aceptación

1. Se lista y selecciona un usuario activo para asociar al pedido.
2. Se selecciona la forma de pago: TARJETA, TRANSFERENCIA o EFECTIVO.
3. Se muestran los productos activos y disponibles para seleccionar.
4. Por cada producto se valida: que exista, que disponible = true y que tenga stock suficiente.
5. Si alguna validación falla después de iniciar la transacción, se hace rollback completo.
6. El subtotal de cada `DetallePedido` se calcula como `precio * cantidad`.
7. El total del Pedido se calcula llamando a `calcularTotal()` (suma de subtotales).
8. El stock de cada producto se reduce al confirmar el pedido.
9. El estado inicial es PENDIENTE y la fecha es la fecha actual.

10. Toda la operación ocurre dentro de una única transacción atómica.

11. Al guardar se muestra: ID generado, total, usuario y resumen de productos con cantidades y subtotales.

HU-17 | Cambiar estado de un pedido

Como: operador del sistema

Quiero: actualizar el estado de un pedido existente

Para: reflejar el progreso de la orden en el sistema

Prioridad: Alta

Story Points: 8

Criterios de Aceptación

1. Se solicita el ID del pedido.
2. Si no existe o está dado de baja, se informa el error.
3. Se muestra el estado actual del pedido.
4. Se permite seleccionar el nuevo estado: PENDIENTE, CONFIRMADO, TERMINADO o CANCELADO.
5. El cambio se persiste correctamente.
6. Se confirma mostrando el ID del pedido y su nuevo estado.

HU-18 | Dar de baja un pedido

Como: operador del sistema

Quiero: dar de baja un pedido incorrecto o cancelado

Para: ocultarlo del sistema sin perder el historial

Prioridad: Alta

Story Points: 6

Criterios de Aceptación

1. La baja es lógica: eliminado = true, el registro permanece en la BD.
2. Si el ID no existe o ya está dado de baja, se informa el error.
3. El stock de los productos NO se restaura.
4. Los DetallePedido permanecen en la BD.
5. Se confirma mostrando el ID y el total del pedido dado de baja.

HU-19 | Consulta JPQL: Pedidos por usuario

Como: operador del sistema

Quiero: ver todos los pedidos activos de un usuario específico

Para: consultar el historial de compras de un cliente

Prioridad: Alta

Story Points: 8

Criterios de Aceptación

1. Se lista y selecciona un usuario activo.
2. Se llama a `pedidoRepo.buscarPorUsuario(idUsuario)`.
3. Se muestra: ID, fecha, estado, forma de pago y total de cada pedido.
4. Si el usuario no tiene pedidos activos, se informa explícitamente.
5. La consulta usa `List<Pedido>` con parámetro nombrado y filtra `eliminado = false`.

HU-20 | Consulta JPQL: Pedidos por estado

Como: operador del sistema

Quiero: filtrar pedidos activos por su estado

Para: gestionar los pedidos pendientes o terminados rápidamente

Prioridad: Alta

Story Points: 8

Criterios de Aceptación

1. Se muestran las opciones disponibles: PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO.
2. Se llama a `pedidoRepo.buscarPorEstado(estado)`.
3. Se muestra: ID, fecha, nombre de usuario y total de cada pedido.
4. Si no hay pedidos con ese estado, se informa explícitamente.
5. La consulta usa `List<Pedido>` con parámetro nombrado y filtra `eliminado = false`.

HU-21 | Reporte: Total facturado

Como: operador del sistema

Quiero: conocer el total acumulado de los pedidos en estado TERMINADO

Para: tener una vista rápida de la facturación del sistema

Prioridad: Alta

Story Points: 5

Criterios de Aceptación

1. Al seleccionar la opción, el sistema obtiene los pedidos con estado TERMINADO y `eliminado = false`.
2. Suma los campos total de todos los pedidos del resultado.

3. Muestra el resultado formateado (ej: Total facturado: \$12500.00).

4. Si no hay pedidos terminados, muestra \$0.00.

La calificación se compone de dos partes. La Parte 2 (Backend JPA/Consola) se evalúa con la rúbrica detallada en la primera tabla (135 puntos). La Parte 1 (Frontend Web) se evalúa con la rúbrica de la segunda tabla (65 puntos). El puntaje total del trabajo es de 200 puntos.

Parte 2 – Backend JPA / Consola (135 pts)

Ítem	Descripción	Puntaje
HU-01: BaseRepository<T>	CRUD genérico correcto, transacciones, Optional, cierre de EntityManager en finally.	18 pts
HU-02: Categoría y ProductoRepo	Extensión correcta, super() con Class<T>, buscarPorCategoría con JPQL tipado y comentario.	12 pts
HU-03: UsuarioRepository	Extensión correcta, buscarPorMail con JPQL tipado, Optional bien manejado, comentario.	8 pts
HU-04: PedidoRepository	Extensión correcta, buscarPorUsuario y buscarPorEstado con JPQL tipado, parámetros nombrados, comentarios.	10 pts
HU-05/06/07: ABM Categorías	Alta con validación de nombre, modificación con campo vacío conservando valor, baja lógica con confirmación, listado.	10 pts
HU-08/09/10: ABM Productos	Alta con selección de categoría, validación precio/stock, modificación, baja lógica con confirmación, listado.	12 pts
HU-11: JPQL Productos/Categoría	JPQL correcto, List, parámetro nombrado, manejo de resultado vacío.	5 pts
HU-12/13/14: ABM Usuarios	Alta con validación de mail único, modificación con campo vacío conservando valor, baja lógica, listado.	10 pts
HU-15: Búsqueda por mail	Llamada correcta a buscarPorMail, manejo del Optional, datos mostrados o mensaje de no encontrado.	5 pts
HU-16: Alta de pedido	Validaciones (disponible, stock), rollback completo ante error, cálculo de subtotales y total, reducción de stock, transacción atómica.	20 pts
HU-17/18: Estado y baja pedido	Cambio de estado con confirmación. Baja lógica sin restaurar stock.	8 pts
HU-19/20/21: Reportes pedidos	JPQL correcto, listados completos, total facturado con formato.	10 pts
Integración general	El proyecto compila y ejecuta sin errores. Menú navegable. Reglas técnicas cumplidas.	7 pts
SUBTOTAL BACKEND		135 pts

Parte 1 – Frontend Web (65 pts)

Ítem	Descripción	Puntaje
FHU-01/02: Autenticación	Login y registro contra usuarios.json, validaciones, persistencia de sesión en localStorage y redirección por rol.	10 pts
FHU-03/04: Catálogo y detalle	Grid de productos con filtro/búsqueda/orden client-side, filtro disponible=true y eliminado=false, detalle con validación de stock.	12 pts

FHU-05/06: Carrito y pedidos	Carrito en localStorage, cálculo de subtotal/envío/total, checkout con forma de pago (TARJETA/TRANSFERENCIA/EFFECTIVO), historial filtrado por usuario.	15 pts
FHU-07/08/09/10: Administración	Dashboard con estadísticas, CRUD de categorías y productos en memoria, gestión de pedidos con cambio de estado y filtros.	18 pts
Estructura, UX y capa de fetch	Estructura de proyecto Vite/TypeScript correcta, capa de fetch a JSON aislada y reemplazable por la API, proyecto ejecuta con npm run dev sin errores.	10 pts
SUBTOTAL FRONTEND		65 pts

PUNTAJE TOTAL DEL TRABAJO: 200 pts (Backend 135 + Frontend 65).

Mínimo para aprobar: 90 pts sobre 200 (equivale al 45% del total; deben aprobarse ambas partes con al menos el 40% de su puntaje cada una).

Nota: Se descuentan 10 puntos por cada error de compilación que impida ejecutar el proyecto.

9. Condiciones de Entrega

9.1 Código fuente

- Comprimir ambos proyectos en un único archivo .zip con el nombre Apellido_Nombre_TPI_Prog3.zip. Debe contener dos carpetas separadas: frontend/ para el proyecto Vite/TypeScript y backend/ para el proyecto Gradle Java.
- El backend debe compilar y ejecutar sin errores desde línea de comandos. El frontend debe ejecutarse con npm install && npm run dev sin errores.
- Cada carpeta debe incluir su propio README.md con descripción, instrucciones de instalación y, en el caso del frontend, las credenciales de prueba para cada rol.
- Los archivos de repositorio deben estar en el paquete repository y la clase Main en com.tp.jp.a. Los archivos .json en public/data/ deben contener datos de prueba coherentes (al menos 2 categorías, 5 productos, 2 usuarios y 2 pedidos).

9.2 Video de presentación (obligatorio)

- Duración: entre 15 y 20 minutos. El video debe cubrir ambas partes del proyecto.
- Cámara encendida durante toda la exposición.
- Audio claro y comprensible.
- El link al video debe dejarse en los comentarios de la entrega en el aula virtual.

Contenido del video:

- Presentación breve del alumno.
- Demostración del funcionamiento — Parte 1 (Frontend): login con rol USUARIO y flujo de compra completo (catálogo, filtros, detalle, carrito, checkout); vista de Mis Pedidos; logout y login con rol ADMIN; recorrido del panel de administración (dashboard, CRUD de categorías, CRUD de productos, gestión de pedidos con cambio de estado).
- Explicación de las decisiones técnicas de ambas partes: cómo se implementó la transacción del alta de pedido y las consultas JPQL (backend); cómo está estructurada la capa de fetch al JSON y cómo se reemplazaría por la API (frontend).

- Demostración del funcionamiento — Parte 2 (Backend): crear al menos 2 categorías, 3 productos, 2 usuarios; crear un pedido completo con al menos 2 productos mostrando cálculo de subtotales y total; cambiar estado a CONFIRMADO y luego TERMINADO; consultar reportes por usuario y total facturado; baja lógica de un producto.

Esta versión del documento corrige errores de código, contradicciones e inconsistencias detectadas en la consigna original. El detalle de cada cambio se resume a continuación.

Sección	Problema original	Corrección aplicada
4.5	buscarPorMail: la variable q se declaraba como List<Usuario> pero createQuery() devuelve un TypedQuery.	Se cambió el tipo a TypedQuery<Usuario> q. El getResultList() sigue devolviendo List<Usuario>.
4.6	buscarPorUsuario tenía un punto sobrante tras Pedido.class) que, con el .setParameter siguiente, generaba un doble punto que no compila.	Se eliminó el punto sobrante; el encadenamiento queda createQuery(...).setParameter(...).getResultList().
4.6	buscarPorEstado cerraba la primera línea con punto y coma y luego intentaba encadenar .setParameter, lo que no compila.	Se quitó el punto y coma para que la sentencia sea un único encadenamiento válido.
F2 / F4.1	usuarios.json declaraba “Sin contraseña en el JSON”, pero el login compara mail y contraseña contra ese mismo archivo: era imposible.	Se agregó el campo password al JSON (solo para verificar el login; no se muestra en UI ni se persiste en el registro). Se nombró el campo en F4.1.
2	Se hablaba de “seis entidades que heredan de Base”, contando a Base (que es @MappedSuperclass) y/o a la interfaz Calculable como entidades.	Se reescribió: una clase abstracta Base, una interfaz Calculable y cinco entidades JPA (Categoria, Producto, Usuario, Pedido, DetallePedido).
2.1	DetallePedido listaba solo cantidad y subtotal, omitiendo las relaciones producto y pedido que sí aparecen en 2.2.	Se agregaron los campos producto (@ManyToOne) y pedido (@ManyToOne) a la descripción de DetallePedido.
2.2	Afirmaba “Todas las relaciones son unidireccionales”, pero Pedido–DetallePedido (mappedBy + setPedido) es bidireccional.	Se aclaró que las relaciones hacia Categoria, Producto y Usuario son unidireccionales y que Pedido–DetallePedido es bidireccional.
2.1	calcularTotal():void podía interpretarse como que retorna el total (total = pedido.calcularTotal()), lo que no compila.	Se aclaró que asigna this.total y no retorna: se invoca pedido.calcularTotal() y luego se lee pedido.getTotal().
F2	Decía “La estructura del JSON debe coincidir con la API”, pero el	Se aclaró el mapeo: IDs planos (categoriaId, idUsuario, idProducto) vs objetos en backend, y

	JSON usa IDs planos y omite createdAt, divergiendo del modelo backend.	que createdAt puede omitirse en los JSON de prueba.
F5.2	El catálogo en F5.2 no mencionaba el filtro disponible/eliminado, aunque FHU-03 (criterio 6) sí lo exige.	Se agregó en F5.2 que solo se muestran productos con disponible = true y eliminado = false, alineando con FHU-03.
HU-03 / HU-11	Criterios truncados: "Se usa List<Usuario> ." y "Se usa List<Producto>," (restos de edición).	Se reescribieron como criterios completos (TypedQuery<Usuario> + getResultList; retorno de List<Producto> sin casteos).

Nota: el snippet de buscarPorCategoria (4.4) ya era correcto (encadenamiento sin punto y coma intermedio) y no requirió cambios de sintaxis.

Correcciones aplicadas en la revisión v2

Esta segunda revisión resuelve inconsistencias técnicas adicionales detectadas tras una auditoría completa del documento:

- 2.1 / 2.2: la colección de Pedido se renombró de detallePedido (singular) a detalles (List), se inicializa en su declaración con new ArrayList<>() y se aclara que mappedBy debe coincidir con el campo pedido de DetallePedido.
- 2.2: la celda de la tabla que clasificaba Pedido-DetallePedido como "Composición unidireccional" se corrigió a "Composición bidireccional", coherente con el párrafo; además se aclara que cascade/orphanRemoval no operan bajo baja lógica.
- 4.2 / HU-01: guardar() ahora usa persist() para altas (id null) y merge() para actualizaciones (id existente). Se aclara que el ID generado debe leerse de la entidad retornada por guardar(), evitando el caso getId() == null al usar merge() sobre una entidad nueva.
- 5.4.1: el alta de pedido usa un único EntityManager; los productos se recuperan con em.find() dentro de esa transacción (quedan gestionados, sin merge() innecesario) y el Pedido nuevo se persiste con persist() (cascade a DetallePedido). Se eliminó la mezcla de entidades de distintos EntityManagers.
- 5.5 / HU-21: el total facturado se formatea con String.format(Locale.US, "\$%.2f", total) para evitar el error de representación de Double, tratando null como 0.
- F5.2: se definió el origen del costo de envío (constante fija del frontend documentada en el README) y la fórmula total = subtotal + envío.
- F5.2: el checkout declara las formas de pago TARJETA, TRANSFERENCIA y EFECTIVO, alineadas con el enum FormaPago del backend.
- F5.1: se aclaró que la validación de contraseña (mínimo 6 caracteres) aplica solo al registro, y que el usuario registrado en esta iteración no persiste en usuarios.json (no podrá reloguearse tras logout hasta integrar el backend).
- Sección 8: la rúbrica de evaluación solo cubría el backend (135 pts) pese a que el frontend es obligatorio (9.1) y se demuestra en el video (9.2). Se agregó la rúbrica del Frontend (65 pts), el puntaje total pasó a 200 pts y el mínimo de aprobación se ajustó proporcionalmente a 90 pts.